

Andmed API-st ja TypeScript

11IT · Veebirakendused · JavaScripti moodul, 2. nädal · konspekt

Kust andmed tulevad?

Eelmisel nädalal kirjutasime andmed ise koodi sisse. Päris veebilehtedel nii ei tehta: ilmateade, uudised ja sinu sõnumid tulevad **serverist**, arvutist kusil internetis. Leht küsib need seal olles ise juurde.

Kujuta ette restorani. Sa ei lähe ise kööki, vaid vaatad menüüd ja tellid. Köök valmistab ja kelner toob toidu lauda. **API** on selline menüü: kokkulepe, mida ja kuidas serverilt küsida saab. Sinu kood tellib, server valmistab ja vastus jõuab tagasi sinu lehele.

JSON: andmed teksti kujul

Ava brauseris see aadress: pokeapi.co/api/v2/pokemon/pikachu. Ekraanile tuleb pikk tekst. See on **JSON**, kõige levinum kuju, milles andmed internetis liiguvad. Lähemal vaatlusel on see peaaegu sama mis eelmise nädala objektid:

```
{
  "name": "pikachu",
  "height": 4,
  "weight": 60
}
```

Erinevus on ainult selles, et JSON-is on ka võtmed jutumärkides. See on lihtsalt tekst, mille kuju on kokku lepitud, ja just see kokkulepe teeb võimalikuks, et iga programm oskab seda lugeda.

fetch: küsi andmed koodiga

Andmete küsimiseks on käsk **fetch**. Kirjuta see konsooli:

```
async function laePokemon() {
  const vastus = await fetch("https://pokeapi.co/api/v2/pokemon/pikachu");
  const andmed = await vastus.json(); // tekst objektiks
  console.log(andmed.name, andmed.height);
}

laePokemon();
```

Kaks uut sõna. **await** tähendab „oota, kuni vastus tuleb“. Päring internetti võtab aega, natuke nagu tellimus köögist, ja ilma ootamata läheks programm tühja taldrikuga edasi. Kus kasutatakse await-i, sinna funktsiooni ette tuleb kirjutada **async**. Käsk `vastus.json()` teeb JSON-tekstist tavalise JavaScripti objekti.

Lõks: kui kirjutad lihtsalt `console.log(fetch(...))`, näitab konsool `Promise {<pending>}`. `Promise` tähendab lubadust: „vastus tuleb, aga veel mitte“. Kui näed seda, oled `await`-i unustanud.

Andmed lehele

Nüüd paneme andmed lehele, täpselt nagu eelmisel nädalal ülesannete nimekirjas.

JSONPlaceholder on harjutamiseks mõeldud API, mis annab näidisülesandeid:

```
async function laeYlesanded() {
  const aadress = "https://jsonplaceholder.typicode.com/todos?_limit=5";
  const vastus = await fetch(aadress);
  const ylesanded = await vastus.json(); // massiiv objektidest
  const loend = document.querySelector("#loend");

  for (const y of ylesanded) {
    const rida = document.createElement("li");
    rida.textContent = y.title;
    loend.append(rida);
  }
}

laeylesanded();
```

Aadressi lõpus olev `?_limit=5` ütleb serverile, et piisab viiest ülesandest.

Vaata päringut oma silmaga

Arendaja tööriistades on vahekaart **Network**. Värskena lehte ja näed seal iga päringut, mida leht teeb. Klõpsa ühel ja vaata nelja asja: aadress, meetod (andmete küsimisel GET), **staatuskood** ja vastuse sisu. Staatuskood ütleb, kuidas läks:

Kood	Tähendus
200	Kõik korras, vastus on käes.
404	Sellist aadressi ei leitud. Kas aadressis on trükiviga?
500	Serveris läks midagi katki. Sinu koodis viga ei pruugi olla.

TypeScript: JavaScript õigekirjakontrolliga

Mäletad lõksu `"5" + 3`? JavaScript andis vastuseks `"53"` ega hoiatanud sind. Või tunnitööd, kus kirjaviga elemendi nimes andis vea alles siis, kui nuppu vajutasid? Väikeses programmis leiad sellised vead kiiresti, suures leiab need kasutaja.

TypeScript on JavaScript, millele on lisatud tüübid. See töötab nagu õigekirjakontroll: sa ütled ette, mis tüüpi väärtus kuhu käib, ja TypeScript joonib vea punasega alla juba koodi kirjutamise ajal.

```
let vanus: number = 17; // vanus peab olema arv
vanus = "seitseteist"; // VIGA: tekst ei sobi arvu asemele

function liida(a: number, b: number): number {
  return a + b;
}

liida(2, "3"); // VIGA: "3" on tekst, mitte arv
```

Koolon ja tüübi nimi on kogu põhiline uus asi. Põhitüübid on number, string ja boolean.

Andmete kuju kirja

Eriti kasulik on TypeScript API andmetega. Sõnaga **interface** kirjeldad, mis kujuga andmed serverist tulevad:

```
interface Ylesanne {
  id: number;
  title: string;
  completed: boolean;
}

const ylesanded: Ylesanne[] = await vastus.json();
console.log(ylesanded[0].titel); // VIGA: välja titel pole olemas
```

Nüüd teab redaktor täpselt, mis väljad igal ülesandel on. Ta pakub neid kirjutades ise välja ja joonib trükkivead alla enne, kui programm üldse käima läheb.

Proovi kõike seda **TypeScript Playgroundis** (typescriptlang.org/play), paigaldada pole midagi vaja. Brauser ise TypeScripti ei mõista: enne käivitamist tõlgitakse see tavaliseks JavaScriptiks ja tüübid kaovad.

Miks see oluline on? Tänapäeval kirjutatakse suurem osa uusi veebiprojekte TypeScriptis ja kui me Reacti juurde jõuame, kirjutame ka meie TypeScriptis.

Sõnastik

Mõiste	Lihtsalt öeldes
server	arvuti internetis, mis hoiab andmeid ja vastab päringutele
API	kokkulepe, kuidas serverilt andmeid küsida: nagu restorani menüü
JSON	tekst kokkulepitud kujul, milles andmed liiguvad
päring	küsimus serverile: „anna mulle need andmed“
await	„oota, kuni vastus tuleb“
TypeScript	JavaScript, mis kontrollib tüüpe ja leiab vead enne käivitamist

Kontrolli ennast

- Mis on API? Seleta restorani näitega.
- Mille poolest erineb JSON JavaScripti objektist?
- Mida teeb await ja miks on vaja async-i?
- Mida tähendab, kui konsool näitab Promise {<pending>}?
- Mida tähendavad staatuskoodid 200 ja 404?
- Mille poolest aitab TypeScript, kui kirjutad y.titel asemel y.title?